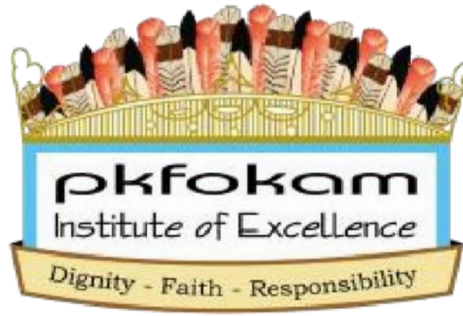


REPUBLIC OF CAMEROON

PEACE-WORK-FATHERLAND



REPUBLIQUE DU CAMEROUN

PAIX-TRAVAIL- PATRIE

Course: C Programming

PKFOKAM INSTITUTE OF EXCELLENCE

PROJECT: WEDDING INVITATION AND GIFT MANAGEMENT SYSTEM

PROJECT REPORT

***GIT HUB REPOSITORY AND PERSON
MANAGEMENT CREATION***

Group Members	Participation	Matricules
Matchim Dzuyim Verenne Celia (Group Leader)	22%	26S031
Djomo Yepdi Evans	19%	26S001
Talla Tchossi Daniella Elsa	19%	26S018
Mekolle Tuatsop Dalhia Aure	21%	26S004
Messi Zang Paul Jacques	19%	26S005

Lecturer: Mr. Mbietieu Amos Mbietieu

Academic Year: 2025/2026

Table of Content

1. Project Overview.....	2
2. Development Tools Used.....	3
3. Work Organisation and task distribution.....	4
4. System Architecture.....	4
5. Functional Modules of the Application.....	5
5.1 Creating Github respository.....	6
5.1.1 Adding collaborators.....	7
5.1.2 Cloning the Respository and first Push.....	7
5.2Creation of the of the c program to manage the guest.....	8
6. Main Difficulties Encountered.....	10
7. Solution Implemented.....	10
8. Phase 1 Summary.....	10
9. References.....	11
10.Gantt project.....	11
11.System design.....	12
11.1 Class Diagram.....	13
11.2 Use Case Diagram.....	14
11.3 Sequence Diagram.....	15
11.4 Deployment Daigram.....	16
11.5 Mnemonic Table.....	16
12.GNU Image Manipulation(GTK) and Comma-Selected Value (CSV).....	17
12.1 GTK.....	17
12.1.1 key Implementation.....	19
12.2 CSV file.....	21

12.2.1 Problem Encountered.....	21
12.2.2 Conclusion.....	22
13. Person and Category Management:Implementation and Test.....	22
13.1 Person management.....	22
13.1.1 Extract of Person Management C-code.....	22
13.1.2 Extract of person management GTK c-code.....	23
13.1.3 GTK successfully.....	24
13.1.4 Executable.....	24
13.2 Category Management.....	25
13.2.1 Extract of Category Management C-code.....	25
13.2.2 Extract Category Management GTK code.....	26
13.2.3 Category ran successfully.....	26
13.2.4 Executable file.....	26
13.2.5 Category Manager Window.....	27
13.2.6 Assignment Page.....	27
14. Combination of Person and Category Management onto a Single.....	27
14.1 Main code with c only.....	27
14.2 Extract of main code with c.....	28
14.3 Mainapp run successfully.....	28
14.4 Executable file.....	28
14.5 Main Window.....	29
15 Conclusion.....	29

1. Project Overview

The **Wedding Invitation and Gift Management System (WIGMS)** is a software application designed to assist in organizing and managing wedding events. Weddings in Cameroon often involve a large number of guests, gift contributions, and multiple social categories, which makes manual management difficult.

The purpose of this system is to simplify wedding planning by providing tools to manage:

- Guest information
- Social categories of guests
- Invitation priority
- Wedding gift tracking

The application is developed using the **C programming language** and follows a **modular programming approach** where each part of the system is implemented as an independent module.

The system aims to improve organization, reduce errors, and help wedding planners efficiently manage guest lists and gifts.

2. Development Tools Used

The following tools were used during the development of the system:

Tool	Purpose
C Programming Language	Main programming language used for system implementation
GCC Compiler	Used to compile and run the C programs
Visual Studio Code / CodeBlocks	Code editor for writing and debugging the program
GitHub	Version control and collaboration platform
Standard C Libraries (stdio.h, ++ stdlib.h,	Used for input/output operations, memory management and string handling

string.h)

These tools help ensure efficient development, collaboration, and proper project management.

3. Work Organisation and task distribution

The group consisted of 5 members. The work organisation depended on assigning responsibilities with respect to skills and the project phases. This approach permitted us to ensure that each section of this report would benefit from dedicated expertise.

Group Member	Main Role	Assigned Tasks
1. Matchim Dzuyim Verenne Celia	Implementation and testing	Common tasks: Project Overview, Development Tools Used, System Architecture, Phase 1 Summary. Individual tasks: GTK, CSV, Test of Person Management, Test of Category Management.
2. Mekolle Tuatsop Dalhia Aure	Documentation and planning	Common tasks: Project Overview, Development Tools Used, System Architecture, Phase 1 Summary. Individual tasks: Report 1, Gantt Project, Report 2, Implementation of Category Management.
3. Talla Tchossi Daniella Elsa	Development (core modules)	Common tasks: Project Overview, Development Tools Used, System Architecture, Phase 1 Summary. Individual tasks: GTK, CSV, Implementation of Person Management.
4. Messi Zang Paul Jacques	System design (UML diagrams)	Common tasks: Project Overview, Development Tools Used, System Architecture, Phase 1 Summary. Individual tasks: Use Case Diagram, Class Diagram.
5. Djomo Yepdi Evans	System modeling and deployment	Common tasks: Project Overview, Development Tools Used, System Architecture, Phase 1 Summary. Individual tasks: Sequence Diagram, Deployment Diagram.

4. System Architecture

The application follows a **simplified modular architecture inspired by the MVC design pattern**. The architecture consists of three main components:

- **Model**

The model contains the core data structures and logic of the program, including:

- Person management
- Category management
- Priority management
- Gift management

▪ **Controller**

The controller handles the flow of the application by managing the interactions between the user interface and the program modules.

▪ **View**

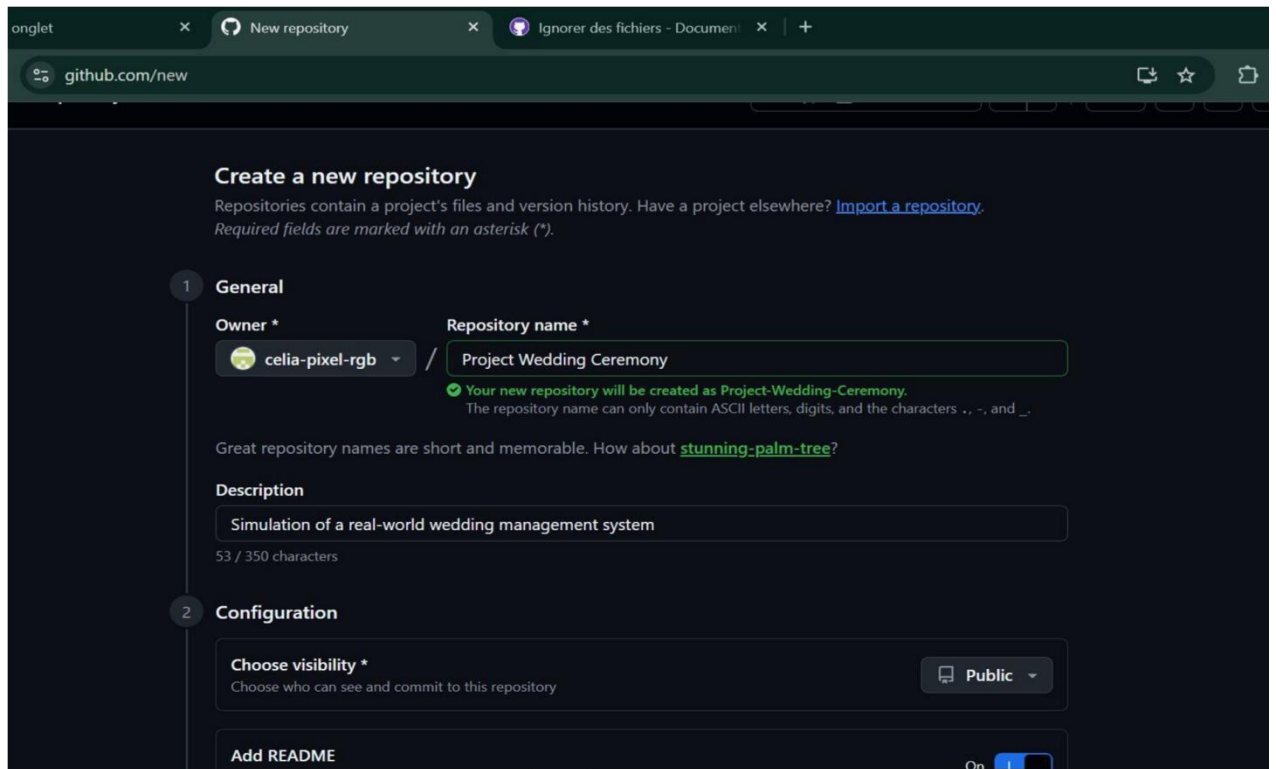
The view represents the interface presented to the user. In this project, the view is implemented through a **console-based menu interface** that allows the user to navigate through the different system functionalities.

This architecture ensures clear separation between data, logic, and user interaction.

5. Functional Modules of the Application

As first stage, we did what we consider as our Module 1: GitHub creation + Personal Management. This module focuses on:

3.1 Creating GitHub repository



The screenshot shows the GitHub 'Create a new repository' page. The 'General' tab is active, showing the repository name 'Project Wedding Ceremony' and the description 'Simulation of a real-world wedding management system'. The 'Configuration' tab is also visible, showing the visibility set to 'Public'.

The group leader created the public repository with the following settings:

Here is the GitHub link to access our repository:

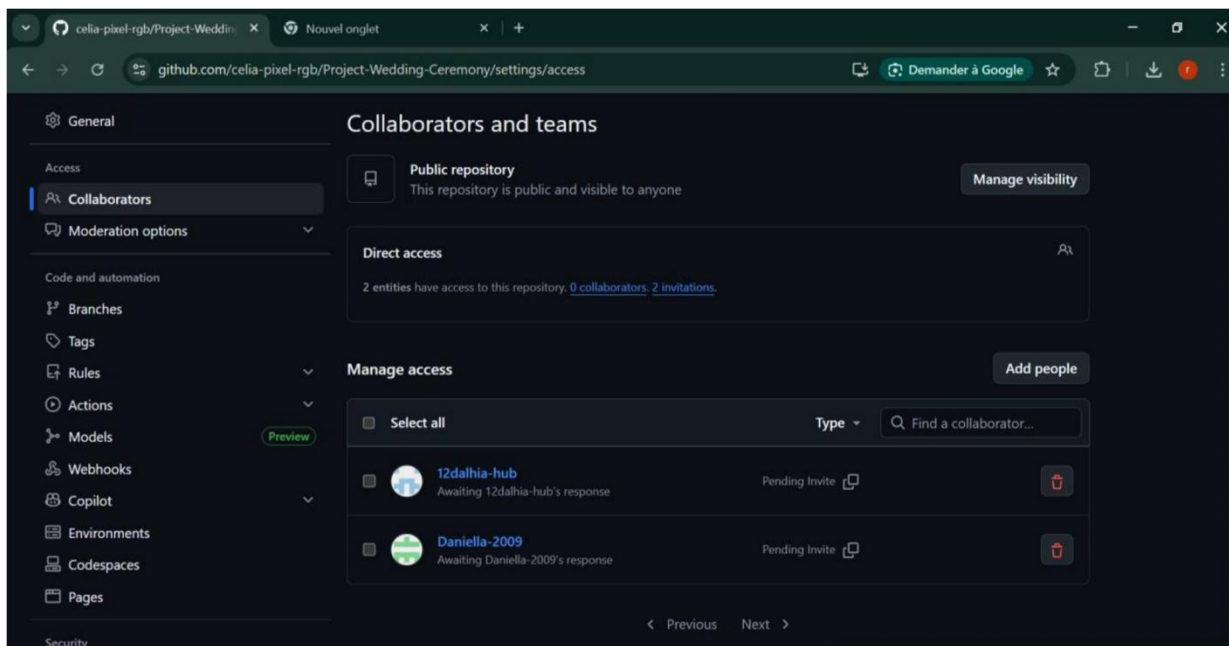
<https://github.com/celia-pixel-rgb/Project-Wedding-Ceremony>

SETTINGS	VALUE CHOSEN
Repository name	Project-Wedding-Ceremony
Visibility	Public
.gitignore	C template selected
Repository link	https://github.com/celia-pixelrgb/Project-Wedding-Ceremony



3.1.1 Adding Collaborators

The remaining group members were added as collaborators through the Setting --> collaborators page of the repository. Each member received an Email invitation from GitHub which they accepted in order to gain push access to the repository.



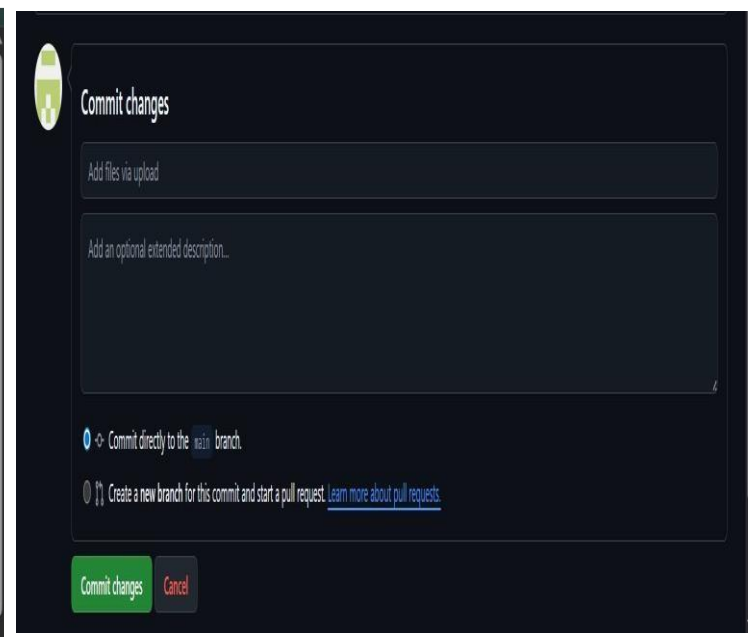
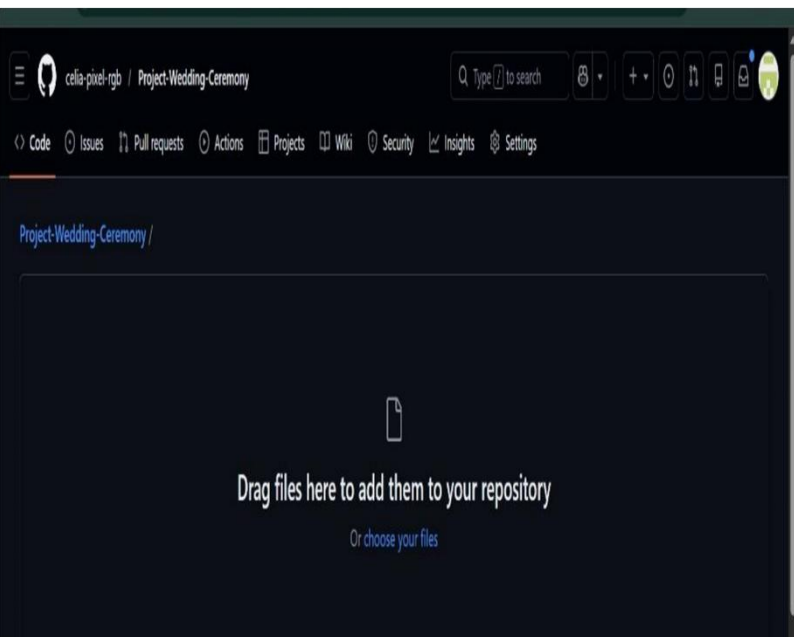
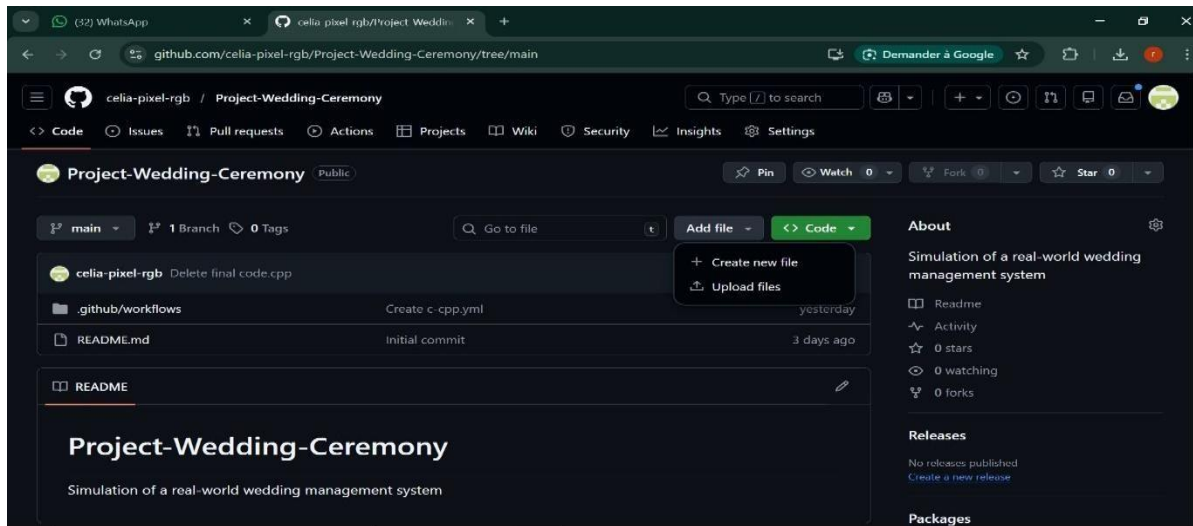
3.1.2 Cloning the Repository and first Push

After accepting their invitation, each member cloned the repository into their local machine. The group leader then performed the first push of the program.

```
Microsoft Windows [version 10.0.17763.1577]
(c) 2018 Microsoft Corporation. Tous droits réservés.

C:\Users\user>git clone https://github.com/celia-pixel-rgb/Project-Wedding-Ceremony.git
Cloning into 'Project-Wedding-Ceremony'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.

C:\Users\user>
```

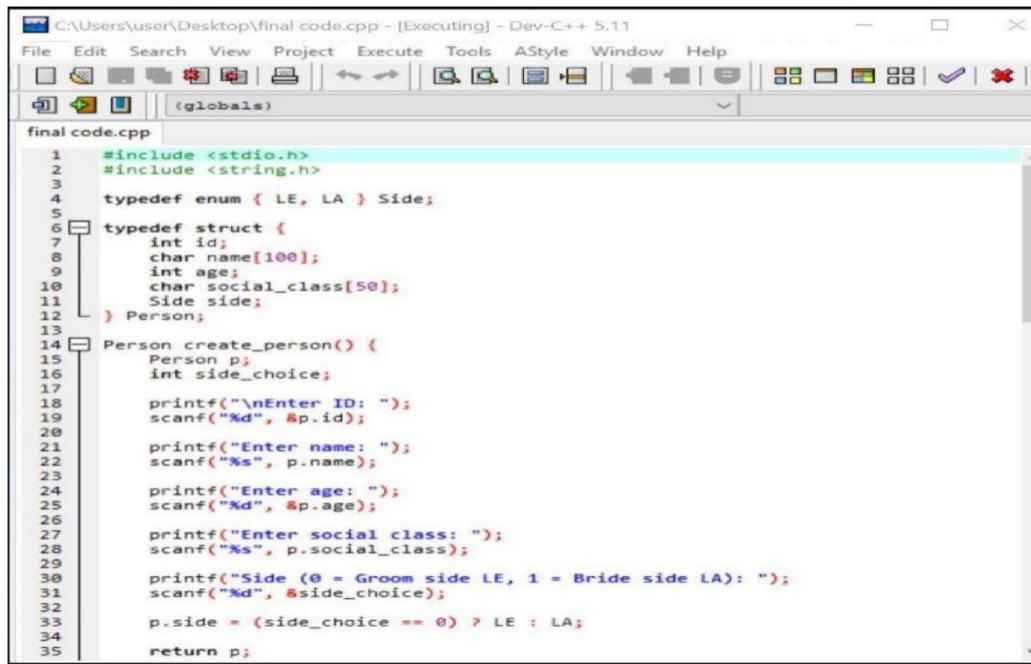



3.2 Creation of the c program to manage the guests

This module manages guest information. It allow the system to:

- **Create a new guest**

Here, we are creating a program that can create persons (guests). Based on the wanted total number of guests entered by the user, we defined a structure in which we register guests base on their name, age, Id, social status and their side (either the bride or the groom).



```

1  #include <stdio.h>
2  #include <string.h>
3
4  typedef enum { LE, LA } Side;
5
6  typedef struct {
7      int id;
8      char name[100];
9      int age;
10     char social_class[50];
11     Side side;
12 } Person;
13
14 Person create_person() {
15     Person p;
16     int side_choice;
17
18     printf("\nEnter ID: ");
19     scanf("%d", &p.id);
20
21     printf("Enter name: ");
22     scanf("%s", p.name);
23
24     printf("Enter age: ");
25     scanf("%d", &p.age);
26
27     printf("Enter social class: ");
28     scanf("%s", p.social_class);
29
30     printf("Side (0 = Groom side LE, 1 = Bride side LA): ");
31     scanf("%d", &side_choice);
32
33     p.side = (side_choice == 0) ? LE : LA;
34
35     return p;

```

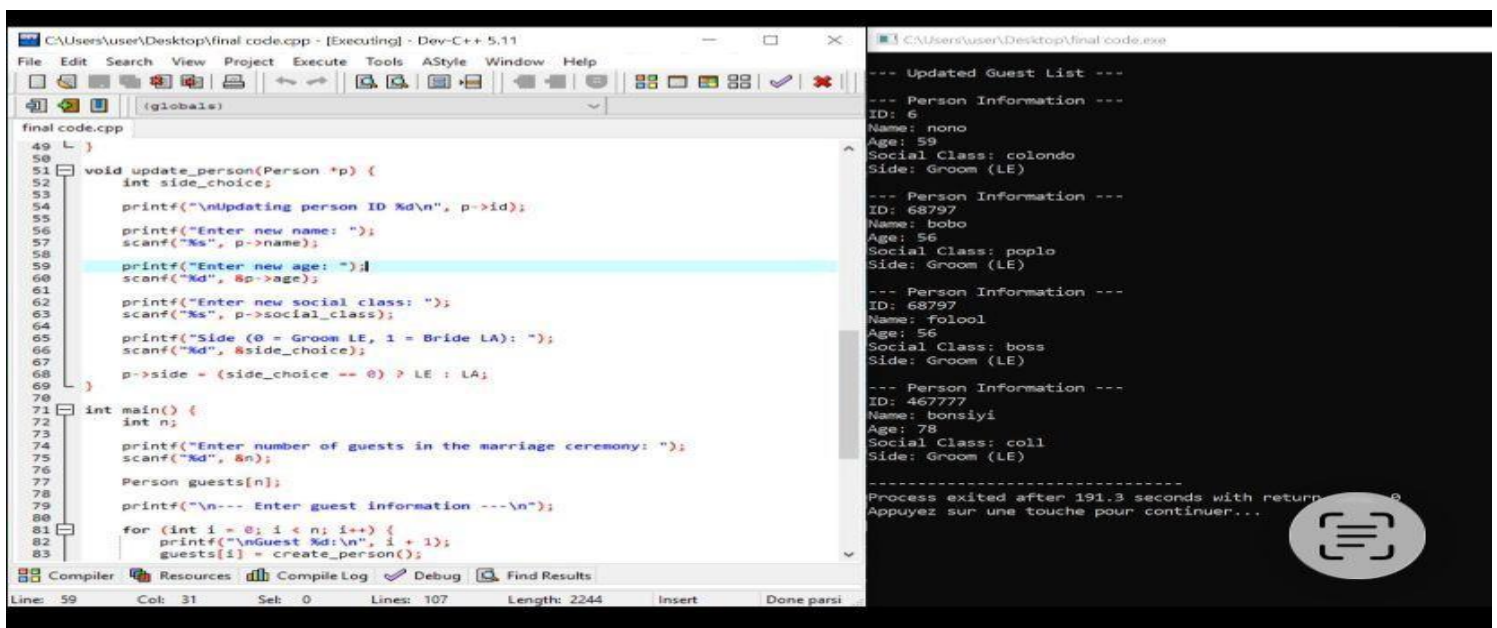
Display the guest

```

void display_person(Person p) {
    printf("\n--- Person Information ---\n");
    printf("ID: %d\n", p.id);
    printf("Name: %s\n", p.name);
    printf("Age: %d\n", p.age);
    printf("Social Class: %s\n", p.social_class);

    if (p.side == LE)
        printf("Side: Groom (LE)\n");
    else
        printf("Side: Bride (LA)\n");
}

```



```

49 }
50
51 void update_person(Person *p) {
52     int side_choice;
53
54     printf("\nUpdating person ID %d\n", p->id);
55
56     printf("Enter new name: ");
57     scanf("%s", p->name);
58
59     printf("Enter new age: ");
60     scanf("%d", &p->age);
61
62     printf("Enter new social class: ");
63     scanf("%s", p->social_class);
64
65     printf("Side (0 = Groom LE, 1 = Bride LA): ");
66     scanf("%d", &side_choice);
67
68     p->side = (side_choice == 0) ? LE : LA;
69 }
70
71 int main() {
72     int n;
73
74     printf("Enter number of guests in the marriage ceremony: ");
75     scanf("%d", &n);
76
77     Person guests[n];
78
79     printf("\n--- Enter guest information ---\n");
80
81     for (int i = 0; i < n; i++) {
82         printf("\nGuest %d:\n", i + 1);
83         guests[i] = create_person();

```

--- Updated Guest List ---

--- Person Information ---
ID: 6
Name: nono
Age: 59
Social Class: colondo
Side: Groom (LE)

--- Person Information ---
ID: 68797
Name: bobo
Age: 56
Social Class: poplo
Side: Groom (LE)

--- Person Information ---
ID: 68797
Name: folool
Age: 56
Social Class: boss
Side: Groom (LE)

--- Person Information ---
ID: 467777
Name: bonsiyi
Age: 78
Social Class: coll
Side: Groom (LE)

Process exited after 191.3 seconds with return code 0
Appuyez sur une touche pour continuer...

Update Guest

6. Main Difficulties Encountered

During the development of the project, several challenges were encountered:

- Understanding the use of **structures in C programming**
- Correctly implementing **alternatives structures, loops and memory management**
- Designing linked list structures for dynamic data
- Managing user input validation
- Debugging errors during program execution

These challenges required careful testing and collaboration among team members.

7. Solutions Implemented

To address the difficulties encountered, the following solutions were implemented:

- Reviewing C programming documentation and tutorials
- Testing each function independently before integration
- Using debugging techniques to identify program errors
- Collaborating through GitHub to track and fix problems

These solutions allowed the team to successfully implement the first module of the project.

8. Phase 1 Summary

The first phase of the project was successfully completed.

The team was able to:

- Understand the project requirements
- Set up the development environment
- Create the project structure
- Implement the **Person Management module**

- Test the system functionality

The project is now ready to proceed to the next phase, which will involve implementing additional modules that we will present later on.

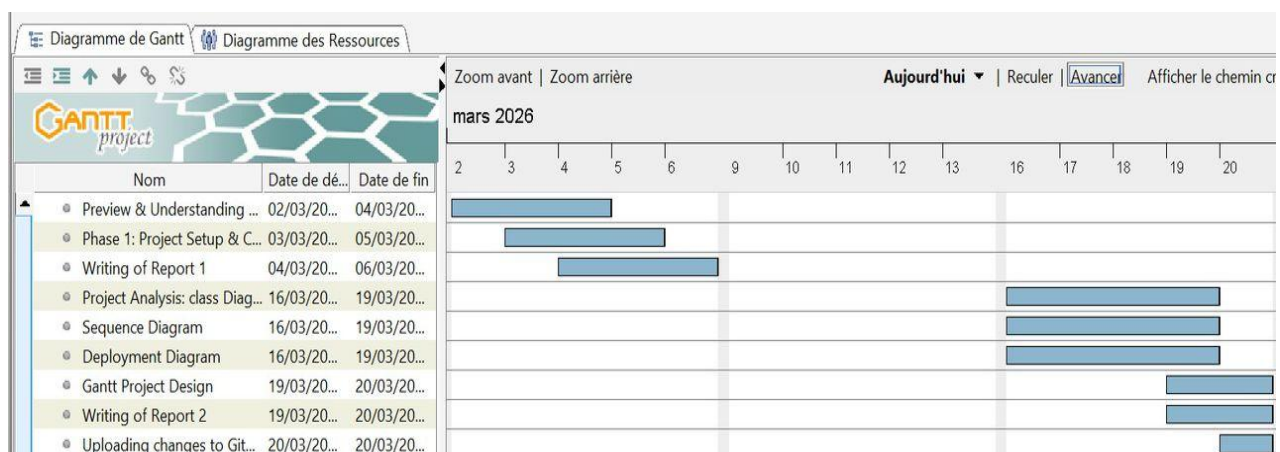
9. References

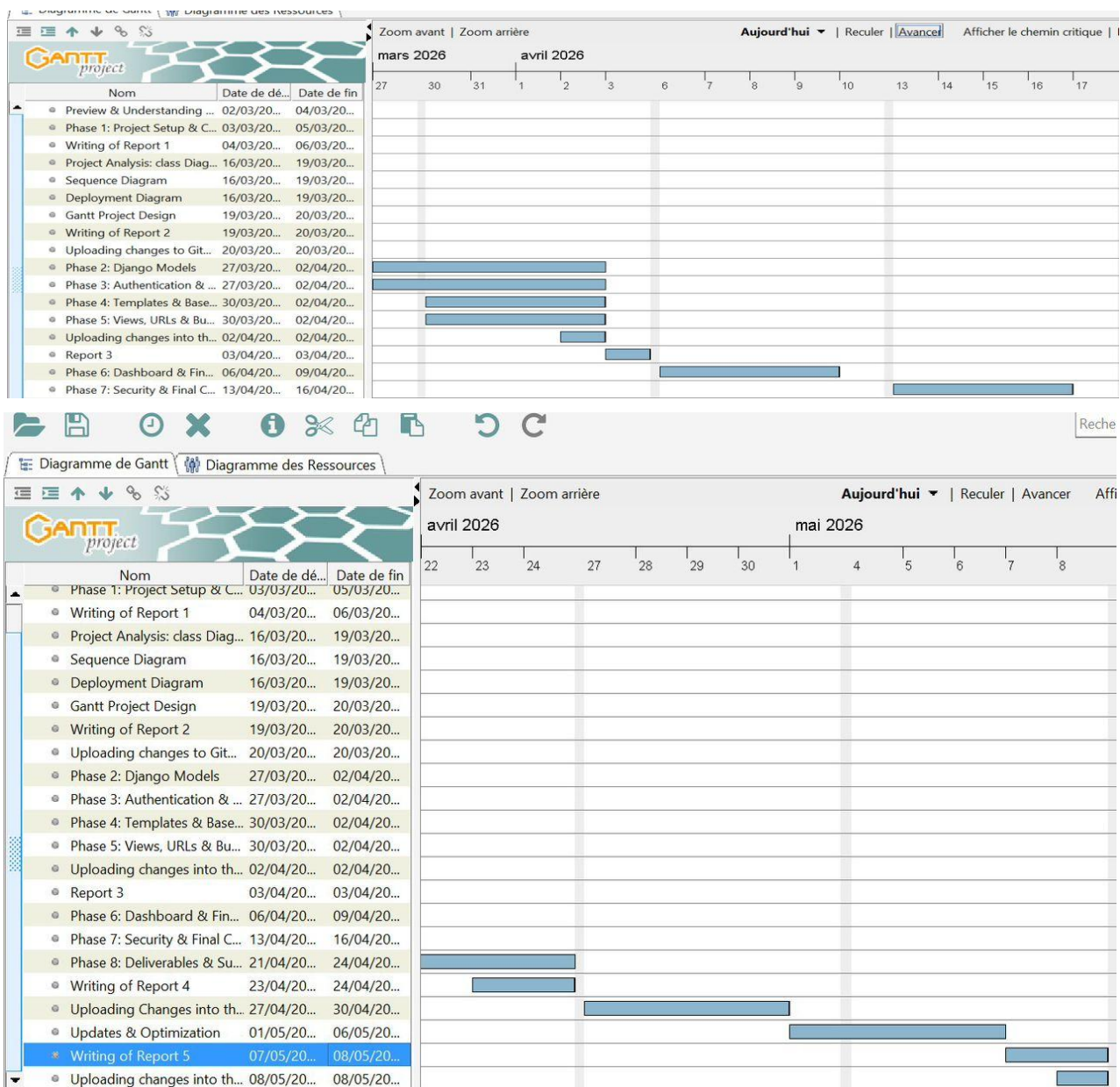
The following references were used during the development of this project:

- **Brian W. Kernighan & Dennis M. Ritchie** – *The C Programming Language*
- **Stephen Prata** – *C Programming: A Modern Approach*
- **ISO C Programming Documentation**
- **GitHub Official Documentation** – <https://docs.github.com>
- Course materials provided by the lecturer

10. Gantt Project

This project outlines the planning, design, and implementation of a software system using structured project management techniques. The Gantt chart was used to organize tasks, timelines, and dependencies efficiently.





The project was successfully structured using a Gantt chart, ensuring proper time management and task coordination.

All phases were completed systematically, resulting in a functional and well-tested application.

11.SYSTEM DESIGN

System design is a critical phase in software development that defines the structure, components, and interactions of a system. It provides a clear blueprint to guide the development process and ensure that all functional and technical requirements are met.

In this section, we will present and explain the key design diagrams used in the project, including the class diagram, sequence diagram, deployment diagram, and use case diagram, to illustrate how the system operates and how its components interact.

11.1 CLASS DIAGRAM

Overview

The WIGMS UML Class Diagram models wedding organization entities using StarUML. A central Wedding class connects nine supporting classes — each with detailed attributes and defined relationships via UML associations and multiplicity notation.

- A class diagram provides a static view of the system, showing classes, attributes, operations, and relationships.
- In WIGMS, it acts as a blueprint for implementation.
- There are four classes; person, parking, gift and category

a. Class Diagram

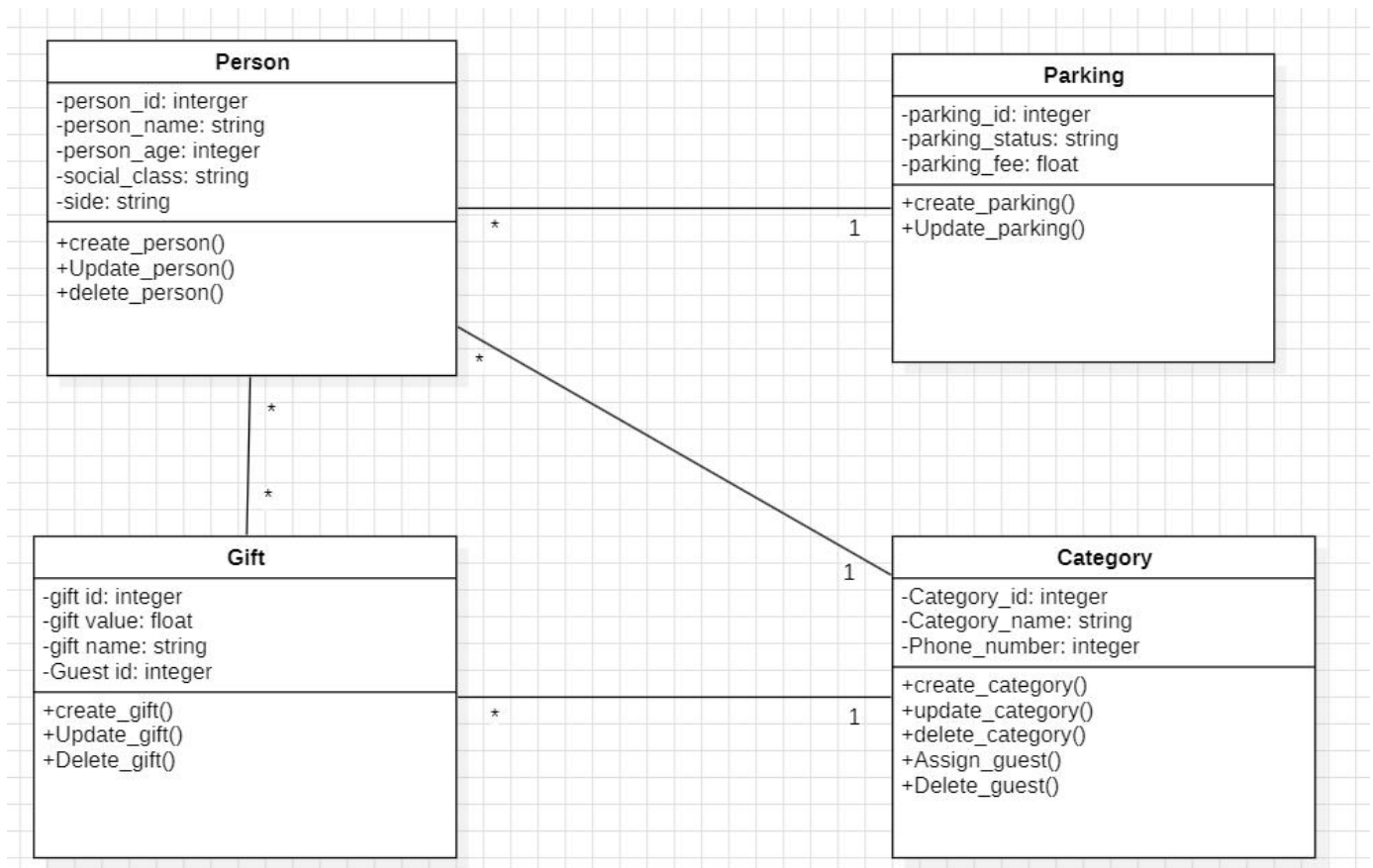


Figure 1: Class Diagram — Wedding Management System (StarUML)

Class	Attributes (private -)	Operations (public +)	Relationship
Person	id, name, age, social_class, side	create, update, delete	* → 1 Parking* → 1 Category* ↔ * Gift
Parking	id, status, fee	create, update	1 → * Person
Gift	id, value, name, guest_id	create, update, delete	* → 1 Category* ↔ * Person
Category	id, name, phone_number	create, update, delete	1 → * Person1 → * Gift

11.2 USE CASE DIAGRAM

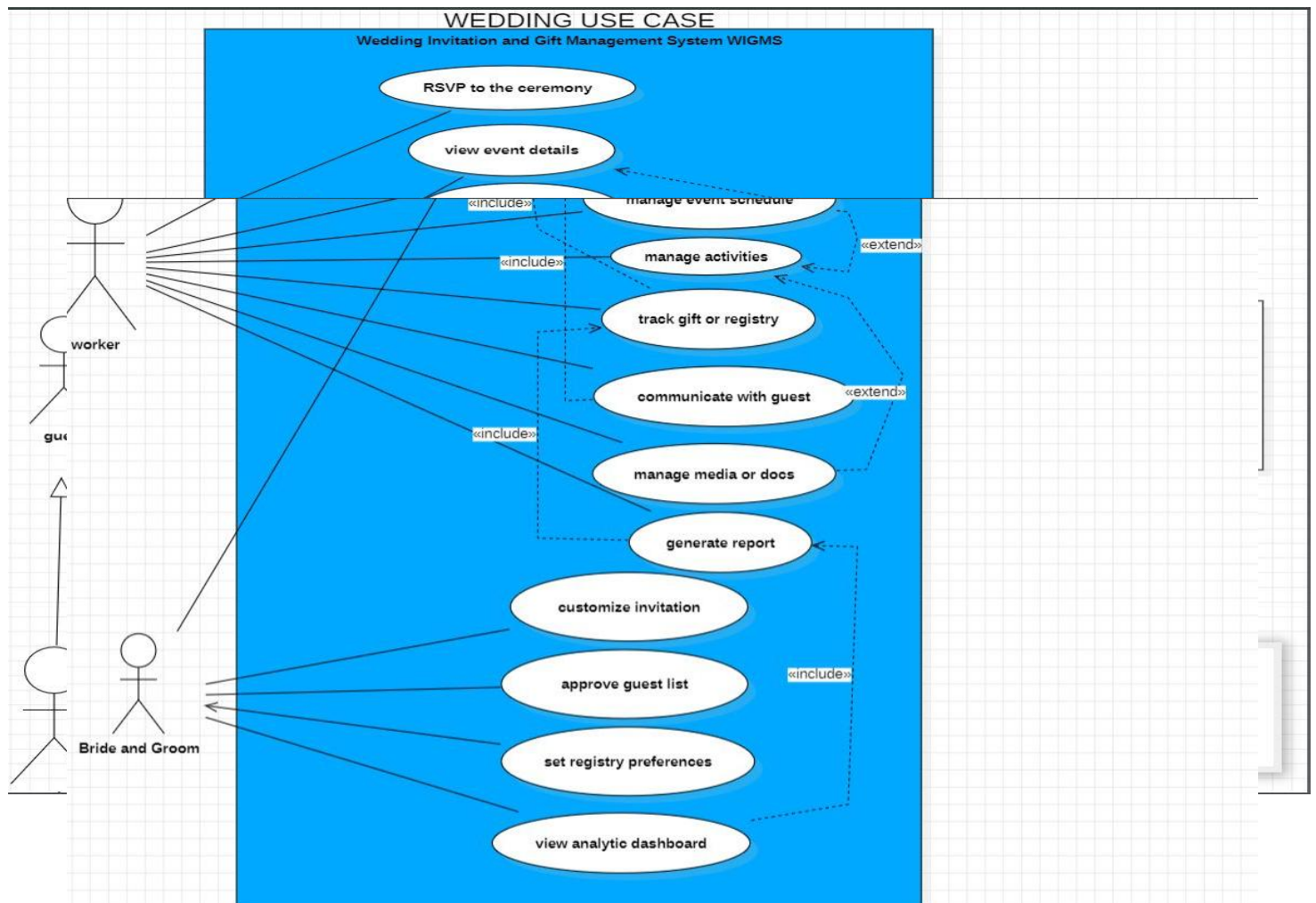


Figure 2: Usecase Diagram — Wedding Management System (StarUML)

11.3 SEQUENCE DIAGRAM

Overview

A sequence diagram illustrates how different components of the wedding-planning system interact over time through ordered message exchanges.

It helps visualize the workflow of actions such as event creation, guest invitations, and RSVP tracking, ensuring processes occur in the correct sequence.

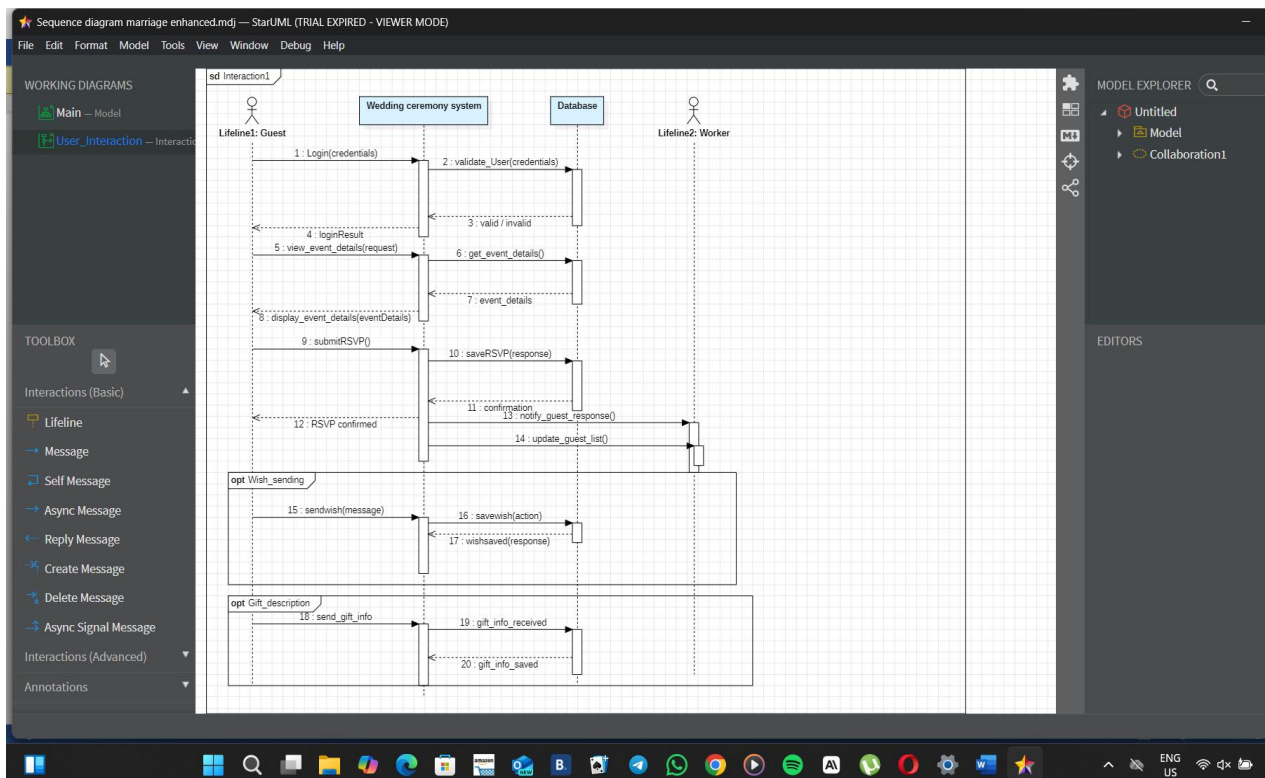


Figure 3: Sequence Diagram — Wedding Management System (StarUML)

11.4 DEPLOYMENT DIAGRAM

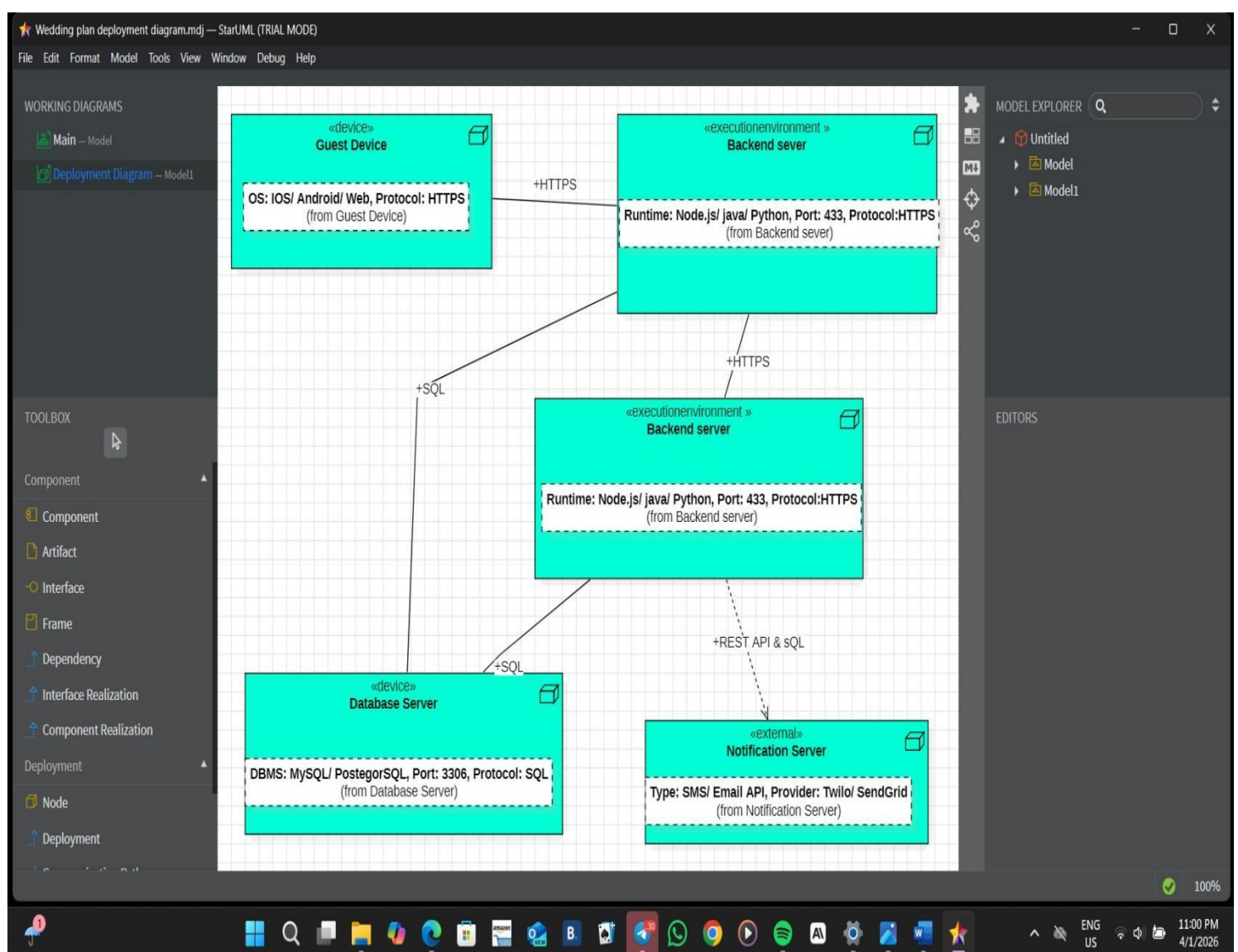


Figure 4: Deployment Diagram — Wedding Management System (StarUML)

11.5 MNEMONIC TABLE

This table will enable any person familiar with c-programming projects who may or may have not participated in the project to better understand the actions going on represented on both the use case and sequence diagrams already illustrated above.

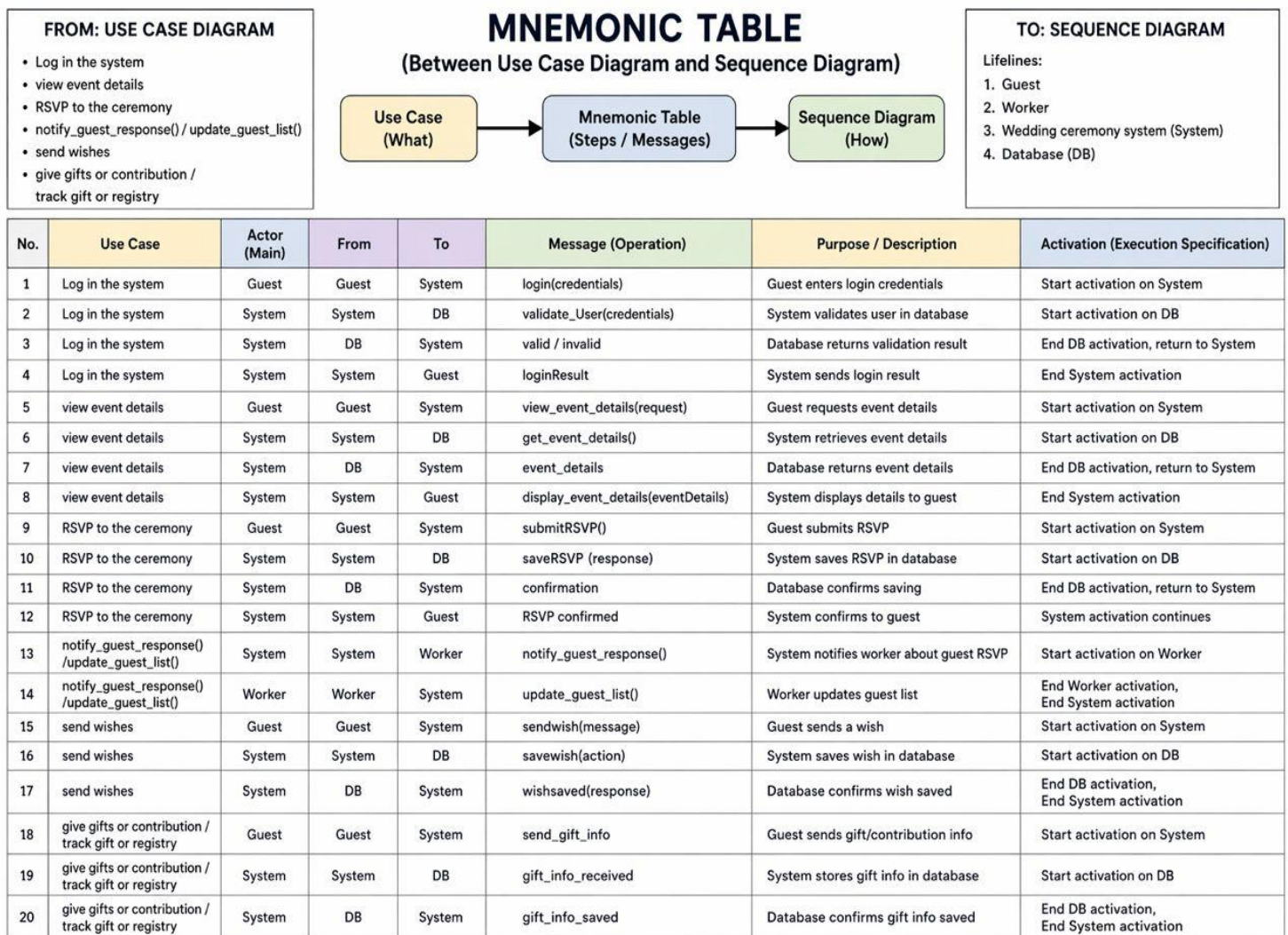


Figure 5: Mnemonic table between the usecase diagram and sequence diagram — Wedding Management System (StarUML)

12. GNU Image Manipulation Program Toolkit (GTK) and Comma-Selected Value (CSV)

12.1 GTK

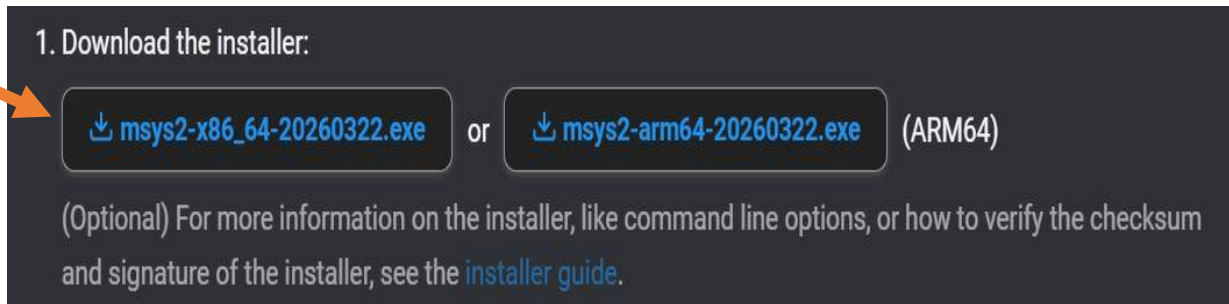
Development of a GTK-based form application written in C for managing wedding guest data. The application collects user input through a graphical interface and stores it in a CSV file.

Program Overview

Download and Installation of GTK 4

The program uses GTK4 to create a graphical interface consisting of input fields for Name, Age, Status,

First, you have to download the installer at msys2.org, selecting the first version for Windows.



- Following this, at the end of the download process, ensure that you store the installer in a folder you will easily remember e.g wedding management ceremony.
- Again, open the “MSYS2 MSYS” terminal and run the command “pacman -Syu”, and press “Y” until you finalize the installations.

```
user@DESKTOP-S24HAFG MSYS ~
$ pacman -Syu
:: Synchronizing package databases...
clangarm64           521.3 KiB   12.4 KiB/s   00:42 [#####] 100%
mingw32              110.7 KiB    7.34 KiB/s   00:15 [#####] 100%
mingw64              459.4 KiB   21.8 KiB/s   00:21 [#####] 100%
ucrt64               538.8 KiB   13.1 KiB/s   00:41 [#####] 100%
clang64              529.2 KiB    9.69 KiB/s   00:55 [#####] 100%
msys                  456.2 KiB   52.0 KiB/s   00:09 [#####] 100%
```

- Still, run the command “\$ pacman -S mingw-w64-ucrt-x86_64-gcc” into the “MSYS2 UCR64” terminal, and validate all the options till the end of the various installations.
- To continue, install GTK still in the “MSYS2 MINGW64” terminal by typing the command “pacman -S mingw-w64-x86_64-gtk4”, and

```

user@DESKTOP-S24HAFG UCRT64 ~
$ pacman -S mingw-w64-ucrt-x86_64-gcc
resolving dependencies...
looking for conflicting packages...

Packages (2) mingw-w64-ucrt-x86_64-gcc-libs-15.2.0-14  mingw-w64-ucrt-x86_64-gcc-15.2.0-14

Total Download Size:    47.67 MiB
Total Installed Size:  248.34 MiB
Net Upgrade Size:       0.07 MiB

:: Proceed with installation? [Y/n]
:: Retrieving packages...
mingw-w64-ucrt-x86_64-gcc-...  546.8 KiB  23.4 KiB/s  33:34 [-----] 1%
mingw-w64-ucrt-x86_64-gcc-...  804.7 KiB  40.2 KiB/s  00:06 [#####] 76%
Total (0/2)                  1351.5 KiB  73.2 KiB/s  10:48 [-----] 2%

```

then install build tools by typing-in the command “`pacman -S mingw-w64-x86_64-pkgconfig mingw-w64-x86_64-make`” in the same terminal.

- Next, in order to ensure that the gcc compiler was well installed, run the command “`gcc --version`” into the “MSYS2 MINGW64” terminal. You should see the version afterwards.

```

M ~
user@DESKTOP-S24HAFG MINGW64 ~
$ gcc --version
gcc (GCC) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

```

- You have now installed your GTK library, you should proceed further with the execution of codes.

NB

Before running a code, you must make sure that it is stored in the same folder as the one containing your GTK library files.

- If the code ran successfully, a “.exe” file appears just below your c-code in the folder.
- In order to run the code, you must open the “MSYS UCR64” terminal and type the command “`gcc file.c -o file.exe $(pkg-config --cflags --libs gtk4)`”

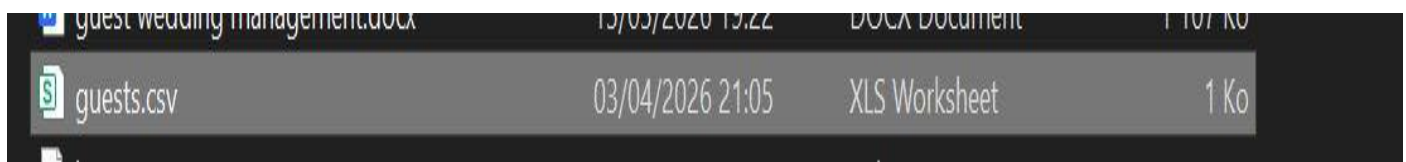

```
user@DESKTOP-S24HAFG UCRT64 ~  
$ gcc person.c -o file.exe $(pkg-config --cflags --libs gtk4)  
  
user@DESKTOP-S24HAFG UCRT64 ~  
$ |
```

13. Key Implementation Details

```
marriage.c x |  
1 #include <gtk/gtk.h>  
2 #include <stdio.h>  
3  
4 // Input fields (global so we can access them easily)  
5 GtkWidget *name_entry;  
6 GtkWidget *age_entry;  
7 GtkWidget *status_entry;  
8 GtkWidget *phone_entry;  
9 GtkWidget *side_entry;  
10  
11 // Function called when button is clicked  
12 void save_data(GtkButton *button, gpointer user_data) {  
13  
14     // Get text from input fields  
15     const char *name = gtk_editable_get_text(GTK_EDITABLE(name_entry));  
16     const char *age = gtk_editable_get_text(GTK_EDITABLE(age_entry));  
17     const char *status = gtk_editable_get_text(GTK_EDITABLE(status_entry));  
18     const char *phone = gtk_editable_get_text(GTK_EDITABLE(phone_entry));  
19     const char *side = gtk_editable_get_text(GTK_EDITABLE(side_entry));  
20  
21     // Open CSV file  
22     FILE *file = fopen("guests.csv", "a");  
23  
24     if (file == NULL) {  
25         g_print("Error opening file\n");  
26         return;  
27     }  
28  
29     // Write data into file  
30     fprintf(file, "%s,%s,%s,%s,%s\n", name, age, status, phone, side);  
31  
32     fclose(file);  
33  
34     g_print("Data saved!\n");  
35 }  
36  
37 // Function that  
38 void activate(GtkApplication *app, gpointer user_data) {  
39  
40     GtkWidget *window;  
41     GtkWidget *grid;  
42     GtkWidget *button;  
43  
44     // Create window  
45     window = gtk_application_window_new(app);  
46     gtk_window_set_title(GTK_WINDOW(window), "Wedding Form");  
47     gtk_window_set_default_size(GTK_WINDOW(window), 400, 300);  
48  
49     // Create layout  
50     grid = gtk_grid_new();  
51     gtk_window_set_child(GTK_WINDOW(window), grid);  
52  
53     // Create input fields  
54     name_entry = gtk_entry_new();  
55     age_entry = gtk_entry_new();  
56     status_entry = gtk_entry_new();  
57     phone_entry = gtk_entry_new();  
58     side_entry = gtk_entry_new();  
59  
60     // Add labels and fields  
61     gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Name"), 0, 0, 1, 1);  
62     gtk_grid_attach(GTK_GRID(grid), name_entry, 1, 0, 1, 1);  
63  
64     gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Age"), 0, 1, 1, 1);  
65     gtk_grid_attach(GTK_GRID(grid), age_entry, 1, 1, 1, 1);  
66  
67     gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Status"), 0, 2, 1, 1);  
68     gtk_grid_attach(GTK_GRID(grid), status_entry, 1, 2, 1, 1);  
69  
70     gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Phone"), 0, 3, 1, 1);  
71     gtk_grid_attach(GTK_GRID(grid), phone_entry, 1, 3, 1, 1);  
72  
73     gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Side"), 0, 4, 1, 1);  
74     gtk_grid_attach(GTK_GRID(grid), side_entry, 1, 4, 1, 1);  
75  
76     // Create button  
77     button = gtk_button_new_with_label("Save");  
78     gtk_grid_attach(GTK_GRID(grid), button, 0, 5, 2, 1);  
79  
80     // Connect button click to function  
81     g_signal_connect(button, "clicked", G_CALLBACK(save_data), NULL);  
82  
83     gtk_window_present(GTK_WINDOW(window));  
84 }  
85  
86 // Main function  
87 int main(int argc, char **argv) {  
88     GtkApplication *app;  
89  
90     app = gtk_application_new("org.example.wedding", G_APPLICATION_DEFAULT_FLAGS);  
91  
92     g_signal_connect(app, "activate", G_CALLBACK(activate), NULL);  
93  
94     return g_application_run(G_APPLICATION(app), argc, argv);  
95 }
```

- ✓ **Global widget pointers** were used to allow access to input fields across functions. This simplifies retrieving user input when the Save button is clicked.
- ✓ **The use of `gtk_editable_get_text()` allows** extraction of user input from each entry field. This is required because GTK widgets do not directly expose their content.
- ✓ **File handling** is implemented using `fopen` in append mode ("a").
This ensures that new data is added without overwriting existing entries.

12.2 Csv file creation



A	B	C	D
0,Matchim,16,VIP,679611233,0,Yes			

The csv file was generated immediately after the GTK form worked, and was tested by inputting arbitrary entries

12.2.1 Problem Encountered and Solution

General Problem

- Difficulty in configuring development tools such as GTK.
- Limited understanding of UML diagrams (sequence, deployment, use case, class).
- Challenges in using StarUML (navigation, relationships, layout).
- Errors in coding such as file handling, compilation, and signal connections.
- Complexity in organizing system interactions and component communication.

12.2.2 General Solution

- Proper installation and configuration of development tools.
- Studying UML concepts and applying correct diagram rules.
- Practicing and exploring StarUML features for better usage.
- Applying debugging techniques and correct coding practices.
- Breaking the project into smaller tasks using structured planning tools like Gantt charts.

12.2.3 Conclusion

The Wedding Ceremony Management System demonstrates how C programming can be combined with project planning tools, system design diagrams, and user interface technologies. The use of GTK and CSV enhances usability and data management, making the system efficient and practical for real-world applications.

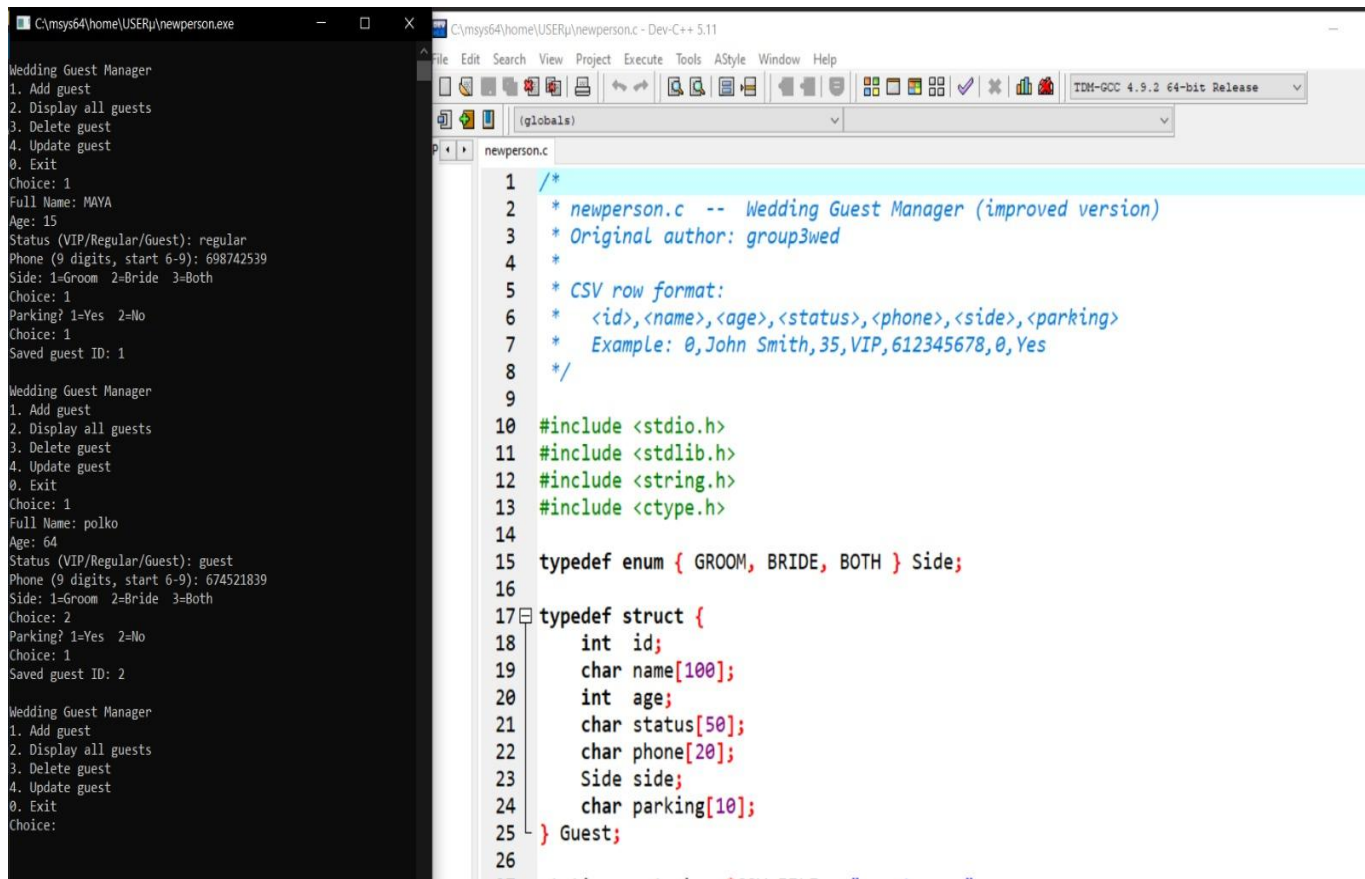
13. PERSON AND CATEGORY MANAGEMENT: IMPLEMENTATIONS & TESTS

13.1 Person Management

This module concerns the registration of the guests at a wedding ceremony, alongside all the operations carried out on them (Entry, Update, Delete). The previous model in the second report being lacking, the following is an update version

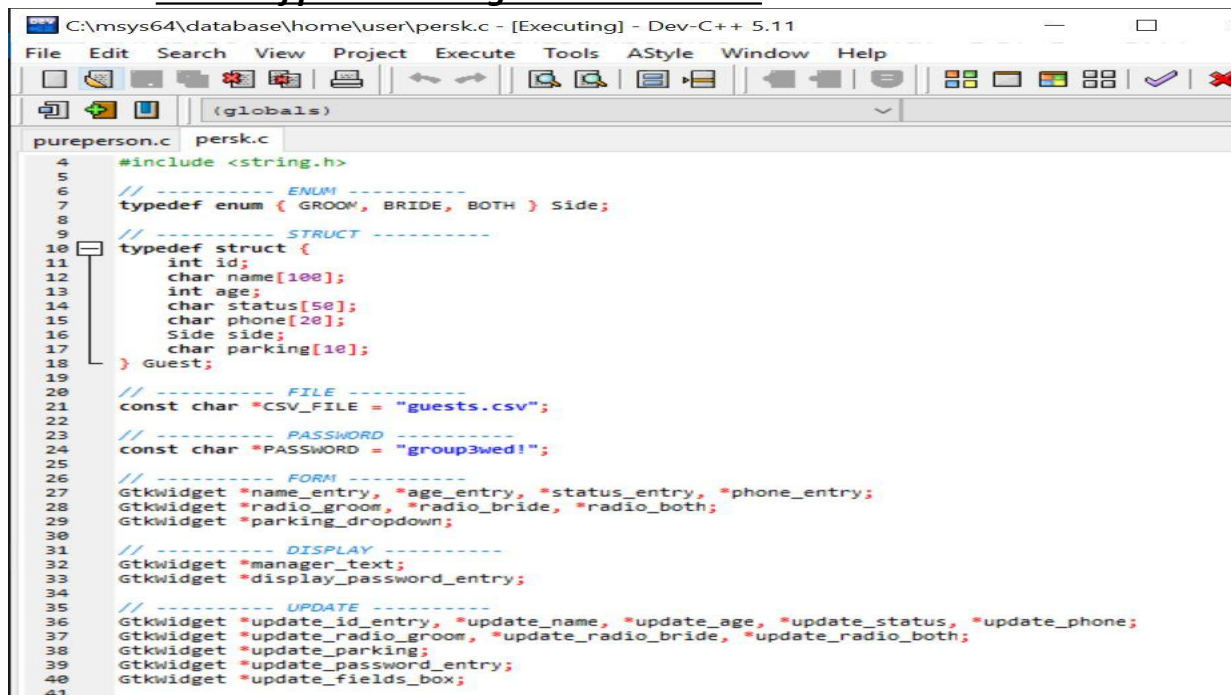
- The c-program was written and thoroughly debugged using dev c++ software

13.1.1 Extract of Person Management C-code



Once written, the c-program was converted into a GTK form using GTK-specific libraries

13.1.2 Extract of person management GTK c-code



The csv file was automatically generated below the gtk person management file immediately the gtk compiled successfully.

13.1.3 GTK ran successfully

```
user@DESKTOP-S24HAFG UCRT64 ~  
$ gcc updatedperson.c -o updatedperson.exe $(pkg-config --cflags --libs gtk4)  
  
user@DESKTOP-S24HAFG UCRT64 ~  
$ gcc persk.c -o persk.exe $(pkg-config --cflags --libs gtk4)  
  
user@DESKTOP-S24HAFG UCRT64 ~  
$ |
```

13.1.4

executable



- Tests were carried out to verify that the Person management form performs its job correctly

(a) Person registration

(b) Person update

Person Manager							
Form	Display	Update	Delete				
Enter password							
Refresh							
ID	Name	Age	Status	Phone	Side	Parking	
0	Matchim Celia	18	VIP	677777777	Both	Yes	

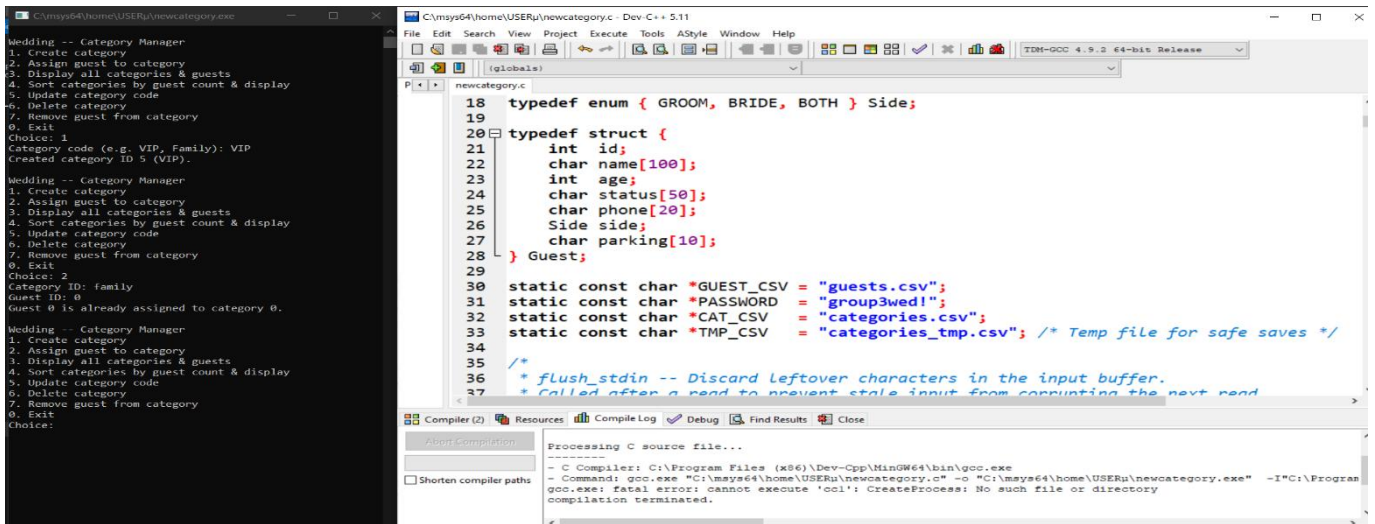
Person Manager			
Form	Display	Update	Delete
ID to update			
Enter password			
Load Guest			

13.2 Category Management

The category management module was implemented in order to allow for creation of different categories at a wedding ceremony.

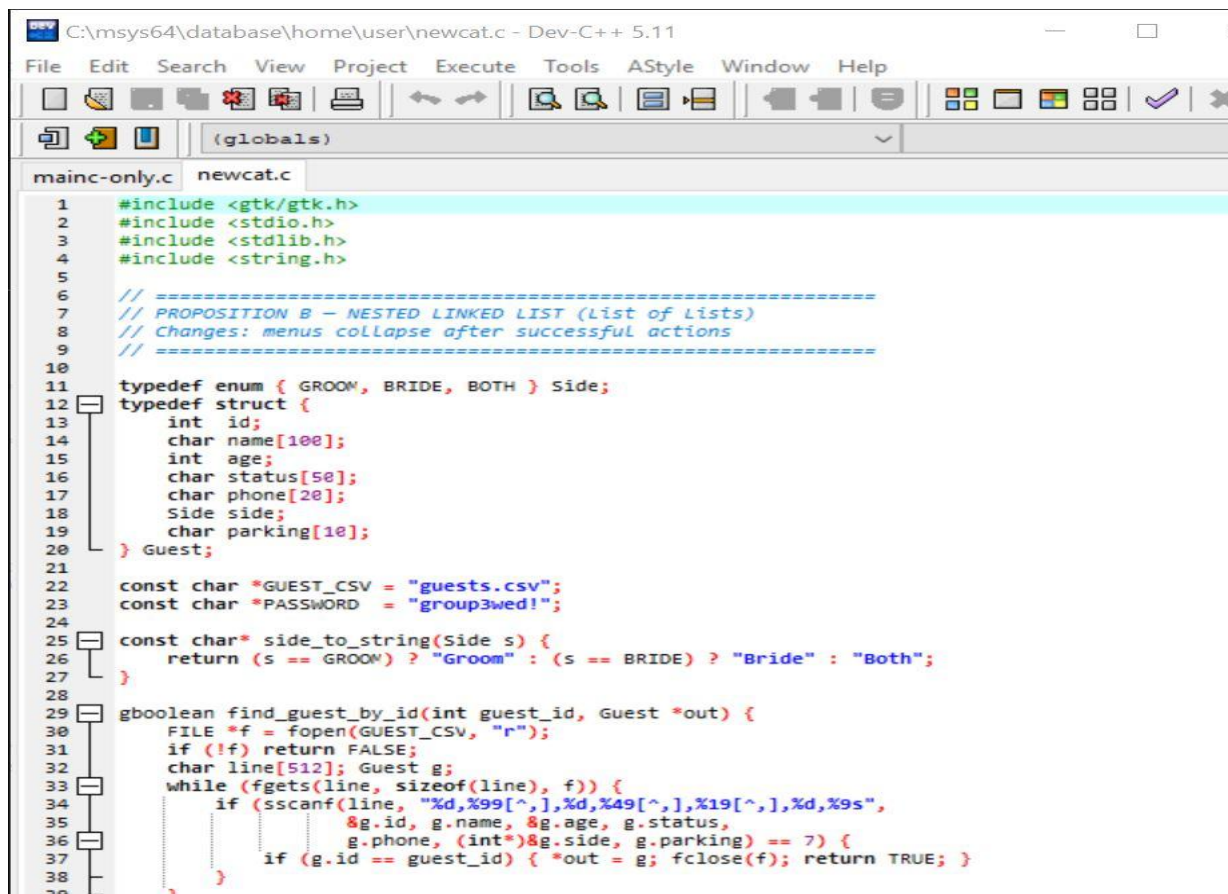
- Just like the person management module, alongside all the operations (Display, Update, Delete, Assign Guests)
- For assigning guests, the code was linked to the person management code, through the ID
- The code was thoroughly debugged using dev c++ software

13.2.1 Extract of Category Management C-code



- Once the category management c program was correctly implemented, the GTK form version was created.

13.2.2 Extract of Category Management GTK code



When properly used, the file is intended

automatically below the category management.

13.2.3 Category ran successfully


```

M ~
user@DESKTOP-S24HAFG UCRT64 ~
$ gcc newcat.c -o newcat.exe $(pkg-config --cflags --libs gtk4)
user@DESKTOP-S24HAFG UCRT64 ~
$ |

```

13.2.4 Executable file



 newcat	03/04/2026 09:42	C Source File	26 Ko
 newcat	03/04/2026 09:42	Application	168 Ko

- Haven properly created the csv file, test were carried out, similarly to those in the person management, to ensure functionality

13.2.5 Category Manager Window

Wedding  Category Manager [Nested List]

New Category

Assign Guest

Display

Update

Delete

? Close

Create a new Category

Code:

Create Category

13.2.6 Assignment Page

Wedding  Category Manager [Nested List]

New Category

Assign Guest

Display

Update

Delete

Password:

Show All

Sort by Guest Count

NESTED LINKED LIST | Categories: 1 | Total guests assigned: 1

(No fixed guest limit per category)

[Category 5 | VIP

]

(1 guest(s))

[GuestID 0] Matchim Celia

| Age: 18 | VIP

| 67777777

| Both

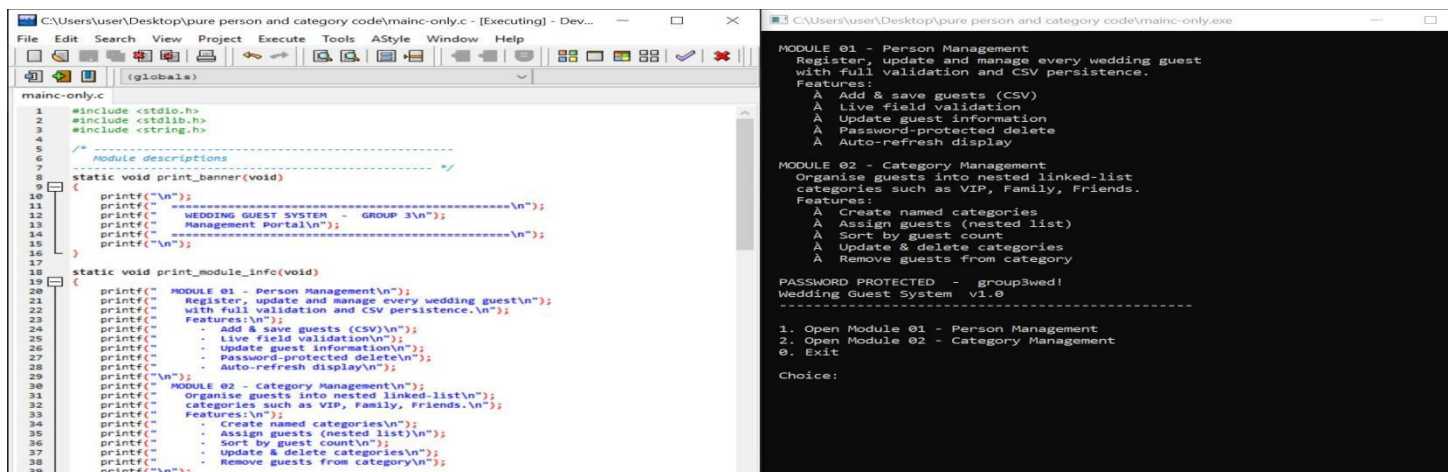
| Parking: Yes

14. COMBINATION OF PERSON AND CATEGORY MANAGEMENT ONTO A SINGLE WINDOW

Haven made the person and category management fully-functional; it is necessary to combine them in a single window where a user has the possibility of choosing which to fill.

- The c-code, mainapp, was written, thoroughly de bugged and converted to a GTK form, and the csv file generated subsequently

14.1 Main code with c only



```
mainc-only.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 /*
6  * Module descriptions
7  */
8 static void print_banner(void)
9 {
10     printf("\n");
11     printf("===== \n");
12     printf(" WEDDING GUEST SYSTEM - GROUP 3 \n");
13     printf(" Management Portal \n");
14     printf("===== \n");
15     printf("\n");
16 }
17
18 static void print_module_info(void)
19 {
20     printf(" MODULE 01 - Person Management \n");
21     printf(" Register, update and manage every wedding guest \n");
22     printf(" with full validation and CSV persistence. \n");
23     printf(" Features: \n");
24     printf("   - Add & save guests (CSV) \n");
25     printf("   - Live field validation \n");
26     printf("   - Update guest information \n");
27     printf("   - Password-protected delete \n");
28     printf("   - Auto-refresh display \n");
29     printf("\n");
30     printf(" MODULE 02 - Category Management \n");
31     printf(" Organise guests into nested linked-list \n");
32     printf(" categories such as VIP, Family, Friends. \n");
33     printf(" Features: \n");
34     printf("   - Create named categories \n");
35     printf("   - Assign guests (nested list) \n");
36     printf("   - Sort by guest count \n");
37     printf("   - Update & delete categories \n");
38     printf("   - Remove guests from category \n");
39     printf("\n");
40 }
```

```
MODULE 01 - Person Management
Register, update and manage every wedding guest
with full validation and CSV persistence.
Features:
A Add & save guests (CSV)
A Live field validation
A Update guest information
A Password-protected delete
A Auto-refresh display

MODULE 02 - Category Management
Organise guests into nested linked-list
categories such as VIP, Family, Friends.
Features:
A Create named categories
A Assign guests (nested list)
A Sort by guest count
A Update & delete categories
A Remove guests from category

PASSWORD PROTECTED - group3wed!
Wedding Guest System v1.0
-----
1. Open Module 01 - Person Management
2. Open Module 02 - Category Management
0. Exit

Choice:
```

- Css was added to the GTK inorder to beautify the window

14.2 Extract of Main code with Css

```
#include <gtk/gtk.h>
#include <stdlib.h>
#include <string.h>

/*
 * CSS - dark navy + gold palette
 */
static const char *APP_CSS =
/* window */
/* window {
 * background-color: #0d1b2e;
 *}
 */
/* Top banner */
.banner {
 * background: linear-gradient(135deg, #091422 0%, #0d1b2e 60%, #112038 100%);
 * padding: 36px 48px 28px 48px;
 * border-bottom: 1px solid rgba(201,168,76,0.25);
 *}
.banner-eyebrow {
 * color: #c9a84c;
 * font-size: 10px;
 * letter-spacing: 4px;
 * font-weight: bold;
 *}
.banner-title {
 * color: #f5f0e8;
 * font-size: 30px;
 * font-weight: 300;
 * margin-top: 6px;
 *}
.banner-sub {
 * color: rgba(245,240,232,0.45);
 * font-size: 12px;
 * margin-top: 4px;
 * letter-spacing: 1px;
 *}
.gold-rule {
 * background: linear-gradient(90deg, transparent, #c9a84c, transparent);
 * min-height: 1px;
 * margin: 16px 0 0 0;
 }
```

14.3 mainapp ran Successfully

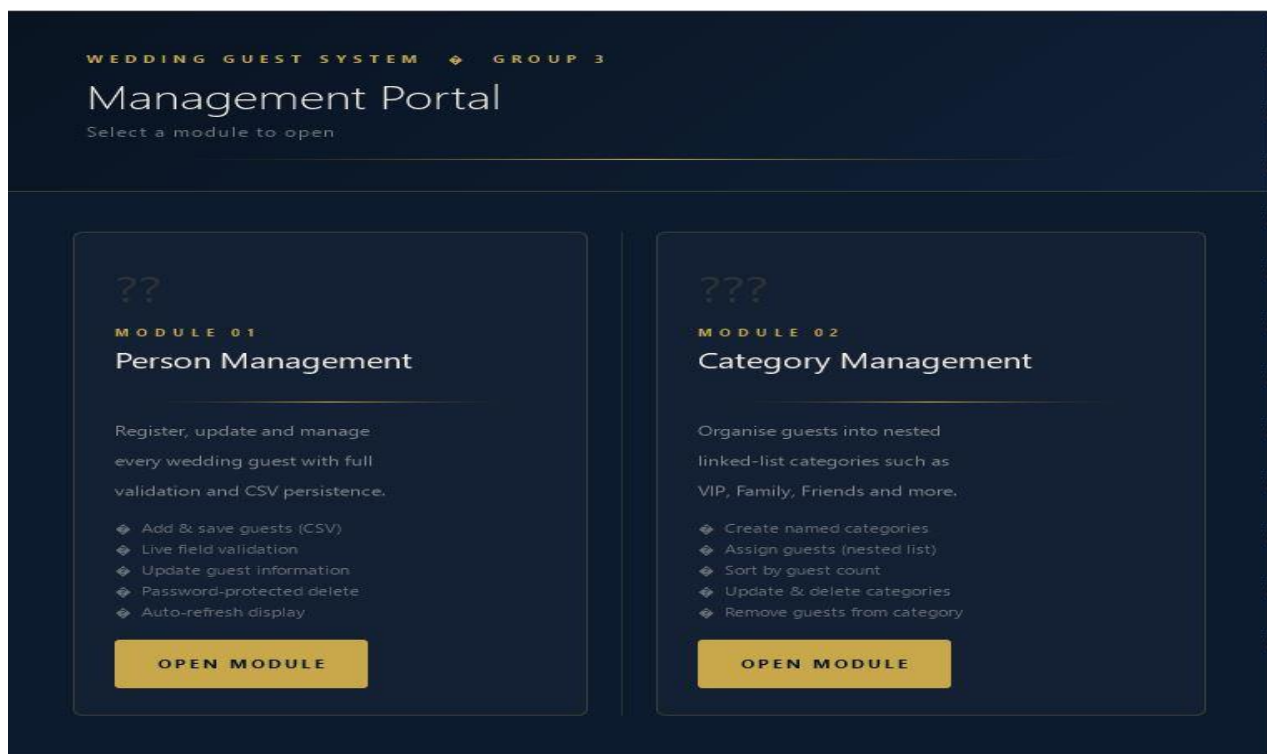
```
user@DESKTOP-S24HAFG UCRT64 ~
$ gcc mainapp.c -o mainapp.exe $(pkg-config --cflags --libs gtk4)
mainapp.c: In function 'on_launch_person':
mainapp.c:160:9: warning: 'gtk_message_dialog_new' is deprecated [-Wdeprecated-declara
tions]
 160 |         GtkWidget *dlg = gtk_message_dialog_new(
      |         ^~~~~~
In file included from C:/msys64/database/ucrt64/include/gtk-4.0/gtk/gtk.h:194,
      | from mainapp.c:16:
C:/msys64/database/ucrt64/include/gtk-4.0/gtk/deprecated/gtkmessagedialog.h:81:12: n
ote: declared here
   81 | GtkWidget* gtk_message_dialog_new          (GtkWindow      *parent,
      |         ^~~~~~
mainapp.c: In function 'on_launch_category':
mainapp.c:181:9: warning: 'gtk_message_dialog_new' is deprecated [-Wdeprecated-declara
tions]
 181 |         GtkWidget *dlg = gtk_message_dialog_new(
      |         ^~~~~~
C:/msys64/database/ucrt64/include/gtk-4.0/gtk/deprecated/gtkmessagedialog.h:81:12: n
ote: declared here
   81 | GtkWidget* gtk_message_dialog_new          (GtkWindow      *parent,
      |         ^~~~~~
user@DESKTOP-S24HAFG UCRT64 ~
$ |
```

14.4 Executable file



	mainapp	03/04/2026 10:33	C Source File	13 Ko
	mainapp	03/04/2026 10:34	Application	153 Ko

14.5 Main Window



NB:

- A

password was used to prevent guests from accessing fields other than registration.
Password: group3wed!

- For being lengthy, full codes could not be integrated in our report. Full codes are available in the GitHub repository at this link: <https://github.com/celia-pixel-rgb/Project-Wedding-Ceremony>
- All the outputs of the codes are present in the repository too.

16. CONCLUSION

With Modules 1 and 2 implemented and optimized, we are ready for the implementation of modules 3 and 4.